The University of Oklahoma

ECE/CS 4613: Computer Architecture

Project 1: MIPS Assembly Programming

Deadline: Nov 13th 11:59 PM, 2026

Introduction to MARS MIPS Simulator

MARS (MIPS Assembler and Runtime Simulator) is a software tool for writing, running, and debugging MIPS assembly programs. It allows you to execute instructions step by step and observe changes in registers and memory.

For this project, you will use **MARS 4.5** to implement and test your MIPS assembly program.

Download: https://dpetersanderson.github.io/download.html

To get started:

1. Visit the website and download MARS 4.5 (Mars4_5.jar).
2. Make sure Java is installed on your computer.
3. Open the downloaded JAR file to launch MARS.
4. Select **File** → **New** to create a new assembly program.
5. Write your MIPS code and save it as Project1.asm.
6. Select **Run** → **Assemble** to assemble your program.
7. Select **Run** → **Go** to execute the program.
8. Check the **Registers** panel to examine the results.
9. You can also use **Run** → **Step** to execute instructions one at a time and observe how register values change.

1. Objective

The objective of this project is to practice MIPS assembly programming using registers, procedures, loops, branches, and memory operations.

You will complete two programming tasks and test your results using the MARS MIPS Simulator. Both tasks must be implemented in a single MIPS assembly program.

2. Project Requirements

Task 1: Implement a Simple Procedure (30 points)

Write a MIPS procedure named addTwo that adds two integers.

Requirements:

- Use 15 and 20 as the two input values.
- Pass the arguments through $a0 and $a1.
- Call the procedure from main using jal addTwo.
- Calculate the sum using an add instruction.
- Return the result through $v0.
- Use jr $ra to return to main.
- Store the returned result in $s5.

Note: You must implement addTwo as a separate procedure and call it from main.

Task 2: Calculate the Sum of an Array (60 points)
Use the following array of 10 integers:
array: .word 12, -5, 23, 7, 0, 18, -9, 31, 4, 15
Write MIPS assembly code to calculate the sum of all array elements.

Requirements:

- Implement the calculation directly in main, without creating a separate procedure.
- Use a loop to process all 10 elements.
- Use lw to load each element from memory.
- Store the final sum in $s3.

Task 3: Testing (10 points)

Run your completed program in MARS 4.5.

- Verify that your program executes successfully without errors.
- Check the final values of $s3 and $s5 in the Registers panel.
- Take one screenshot showing both register values.
- You may display register values in either decimal or hexadecimal.

3. Starter Code

Copy the following code into MARS and save it as Project1.asm.
Complete the sections marked TODO. Do not remove the provided initialization or program termination instructions.

```
.data
array: .word 12, -5, 23, 7, 0, 18, -9, 31, 4, 15
size:  .word 10

.text
.globl main

main:
    # Task 1: Call addTwo

    # TODO:
    # 1. Set $a0 = 15 and $a1 = 20
    # 2. Call addTwo using jal
    # 3. Store the returned result in $s5


    # Task 2: Calculate the array sum
    # Write your loop directly in main

    # Provided initialization
    la   $s0, array
    lw   $s1, size

    # TODO:
    # 1. Initialize the loop counter and sum
    # 2. Use lw and a loop to add all elements
    # 3. Store the final sum in $s3


    # End program
    addi $v0, $zero, 10
    syscall


# Procedure: addTwo
addTwo:
    # TODO:
    # 1. Add $a0 and $a1
    # 2. Store the result in $v0
    # 3. Return using jr $ra
```

Notes about the starter code:

- .data defines the array and its size in memory.
- .text identifies the program's instruction section.
- la $s0, array loads the starting memory address of the array into $s0. This instruction is provided; you do not need to implement it.
- lw $s1, size loads the number of elements into $s1.
- The instructions at the end of main terminate the program in MARS. Do not modify them.

4. Submission Requirements

Submit the following two files through Canvas:

1. Project1.asm

Your completed MIPS assembly program containing both tasks. Include comments explaining the main parts of your implementation.

2. Project1.pdf

A one-page PDF containing a screenshot of the MARS Registers panel showing the final values of $s3 and $s5.

5. Grading Rubric

Component	Points
Task 1: Procedure Implementation	30
Task 2: Array Sum	60
Task 3: Testing and Screenshot	10
Total	**100**

6. Additional Instructions

- Include comments explaining the main parts of your code.
- You may use MIPS pseudo instructions supported by MARS.
- You may refer to the MIPS Assembly Quick Reference.